Take-Home Exercise: Artifact Access Audit Service

Cloudsmith Engineering

Time Box	Approximately 2 hours
Language	Python (framework and library choices are yours)
Deliverable	Public GitHub repository
AI Use	Allowed — you must be able to explain and defend anything you submit

Overview

Cloudsmith is a cloud-native artifact management platform. Our customers use Cloudsmith to store, distribute, and secure software packages across their organisations. One important capability is audit logging: tracking who accessed what, and when.

In this exercise, you will build a small service that ingests artifact access events, stores them with proper tenant isolation, and exposes a query API.

The Problem

You have been provided with a file called events.jsonl. It contains artifact access events across multiple tenants. Each line is a JSON object representing a single event.

The data is intentionally imperfect, as real-world event data tends to be. You should expect to encounter:

- Duplicate events
- Events arriving out of chronological order
- Malformed entries

What to Build

Build a Python service that:

1. Ingestion

- Reads event data from events.jsonl
- Handles duplicates and malformed entries appropriately

2. Tenant Isolation

- Stores events partitioned by tenant
- One tenant must never be able to see another tenant's data

3. Query API

Expose an API that supports filtering by:

- Tenant (required)
- Time range
- Action type (download, upload, delete)

- Package name

4. Retention Policy

- Events older than a configurable threshold should be cleaned up

What to Deliver

Working Code

- Create a public repository in your GitHub profile
- Tests for the important behaviour

README

Your README should cover:

- How to set up and run the service
- Your approach and any key decisions
- Trade-offs you made or shortcuts you took
- What you would do differently with more time

Guidelines

Please time-box this to approximately 2 hours. **We value a clean, well-scoped solution over a feature-complete one.**

It is completely fine to stub or simplify areas and explain what you would do with more time — that is more useful to us than extra polish.

There are no bonus points for exceeding the time box or adding features that weren't asked for.

What We're Looking For

Area	Description
Code structure	How you approach organising and structuring the codebase
Data handling	Appropriate handling of duplicates, malformed entries, and ordering
Tenant isolation	Correct and robust separation of tenant data
API design	Sensible, clean API design choices
Tests	Tests that cover meaningful behaviour, not just coverage